Traceable Harmonization of Ocean Data for ML Readiness

Complete Pipeline Story: From Raw Source to ML-Ready Zarr

Nalluraj Babu

Capstone Project 2026

What this project does: Takes 3 ocean temperature datasets from 3 different institutions, each with different variable names, different units, and different file structures, and combines them into a single, clean, ML-ready Zarr store where every single data point can be traced back to its exact source file, original variable, and the transformation applied to it.

What You Will Learn in This Document

Chapter 1: The Problem: Why Can't We Just Merge the Files?

Chapter 2: Meet Our 3 Real Datasets (from ERDDAP)

Chapter 3: Stage 1 - Ingestion: Read Without Touching

Chapter 4: Stage 2 - SHA-256 Hashing: Fingerprinting Every File

Chapter 5: Stage 3 - Preprocessing: Making the Grid Manageable

Chapter 6: Stage 4 - Transformation: Converting All Units to Kelvin

Chapter 7: Stage 5 - Harmonization: One Schema to Rule Them All

Chapter 8: Stage 6 - Writing to Zarr: ML-Optimized Storage

Chapter 9: Stage 7 - Verification: Proving Nothing Was Tampered

Chapter 10: The Complete Pipeline Flow (Visual)

Chapter 11: Re-Derivability: Change Config, Not Code

Chapter 12: What the ML Engineer Gets (Final Output)

Chapter 13: The Guiding Question Answer

Every chapter shows: what happens, why it happens, how the data looks before and after, and what code/config drives it. No stone is left unturned.

Chapter 1: The Problem

Why Can't We Just Merge the Files?

Imagine you are a machine learning engineer. You want to build a model that predicts ocean temperature in the Northeast Pacific. You find 3 datasets from 3 different institutions. Great! But when you open them, you discover a nightmare:

Dataset 01 (Ocean Networks Canada) calls temperature **"Temperature"** and stores it in **Kelvin**.
Dataset 02 (NOAA CoastWatch) calls it **"sst"** and stores it in **Fahrenheit**.
Dataset 03 (OOI) calls it **"sea_water_temperature_01"** and stores it in **Celsius**.

Three names. Three units. Three file structures. You cannot just concatenate these. If you train an ML model on raw values, it would think 275 Kelvin, 57 Fahrenheit, and 5 Celsius are wildly different temperatures when they are actually measuring the same thing.

THE PROBLEM: Same Ocean, Three Different Languages

Aspect	Dataset 01 (ONC)	Dataset 02 (NOAA)	Dataset 03 (OOI)
Temp Variable	Temperature	sst	sea_water_temperature_01
Temp Units	Kelvin (K)	Fahrenheit (deg F)	Celsius (deg C)
Structure	Tabular (row dim)	4D Grid (t x alt x lat x lon)	Tabular (row dim)
Depth Info	depth (meters, 1966m)	altitude dim (size=1)	z (altitude +up, all 0.0)
Data Size	1,020 rows	63,374,400 grid cells	4,321 rows
Platform	Seafloor sensor	Satellite (MODIS Aqua)	Cabled seafloor sensor
Institution	Ocean Networks Canada	NOAA CoastWatch	OOI

Figure 1: The heterogeneity problem across our 3 real ERDDAP datasets

This project solves this problem with a config-driven pipeline that reads the YAML configuration file, figures out what each dataset calls its variables, what units they are in, and how to convert them all to one common schema (Kelvin, meters, WGS84, UTC). And it does this while keeping a complete trail of every transformation, so any data point can be traced back to its exact origin.

Chapter 2: Meet Our 3 Real Datasets

Downloaded from ERDDAP (April 2026)

All 3 datasets are real ocean data downloaded from ERDDAP servers. They come from the Northeast Pacific Ocean and measure sea water temperature, but each one is structured completely differently:

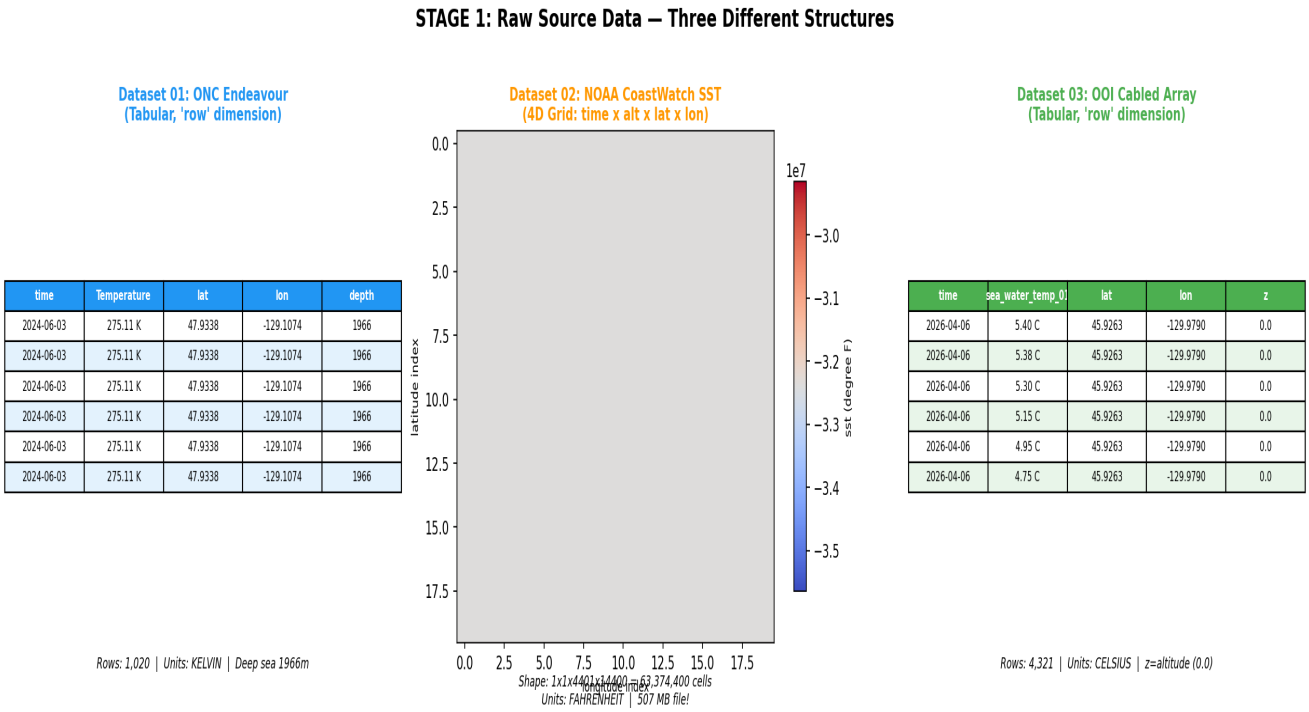


Figure 2: Raw source data as it arrives from ERDDAP - three completely different structures

Dataset 01: Ocean Networks Canada - Endeavour South

This is from a fixed seafloor sensor sitting at 1,966 meters depth on the Endeavour hydrothermal vent ridge. It is a simple table: each row is one observation with a timestamp, a temperature reading, and the fixed coordinates. The unusual thing: it stores temperature in **Kelvin** (most ocean sensors use Celsius). It has **1,020 rows** and the dimension is called "row" (not "obs" or "time").

Dataset 02: NOAA CoastWatch - MODIS Aqua Satellite SST

This is satellite-derived sea surface temperature from NASA's MODIS Aqua sensor. It is fundamentally different: instead of single-point observations, it is a **4D grid** with dimensions (time=1, altitude=1, latitude=4,401, longitude=14,400). That is **63.4 million grid cells** covering the global ocean! The file is 507 MB. And the units are in **Fahrenheit**, which is extremely unusual for ocean science data. Many cells contain NaN (land, clouds) or fill values (-32,767). We need to: squeeze the altitude dimension, spatially subset to just the NE Pacific, flatten the grid to rows, convert Fahrenheit to Kelvin, and remove all the garbage values.

Dataset 03: OOI Regional Cabled Array

From the Ocean Observatories Initiative's cabled seafloor network near Axial Seamount. Temperature is called "sea_water_temperature_01" (sensor #1 of 3). Units are **Celsius**. Instead of a "depth" column, it has "z" which is altitude (positive up = 0.0 at seafloor). It has **4,321 rows** and 13 data variables (we only need 1 temperature + coordinates). We need to: convert Celsius to Kelvin, negate z to get depth, and drop 8 unnecessary QC variables.

Chapter 3: Stage 1 - Ingestion

Read Without Touching the Source

The first rule of this pipeline: **source files are IMMUTABLE**. We never, ever modify the original .nc files. The ingestor module does three things:

Step 1: Compute a SHA-256 hash of the file on disk BEFORE reading it. This creates a unique "fingerprint" of the original file. If even one byte changes later, the hash will be completely different.

Step 2: Open the NetCDF file with xarray using lazy loading. For small files (DS01, DS03), the data loads quickly. For the 507 MB NOAA file (DS02), lazy loading means we do NOT load 63 million cells into RAM yet, we just create a reference to the file.

Step 3: Register the file in the Source Manifest and create a Provenance Record that tracks which institution produced this data, what platform (sensor/satellite), and what region it covers.

Why lazy loading matters for DS02: If we loaded all 507 MB into memory immediately, it would expand to several GB of uncompressed float64 arrays and likely crash the computer. Instead, the harmonizer will first squeeze and spatially subset the data, so only the small NE Pacific subset ever enters RAM.

```
Code: ds = xr.open_dataset(filepath) (no .load() call). The file hash is:
compute_file_sha256(filepath) which reads the file in 8 KB chunks and feeds them to Python's
hashlib.sha256().
```

Chapter 4: Stage 2 - SHA-256 Hashing

Fingerprinting Every File and Every Step

SHA-256 is a cryptographic hash function. You give it any data (a file, an array, a string) and it produces a fixed 64-character hexadecimal string. Two key properties make it perfect for data integrity:

Property 1 - Deterministic: The same input ALWAYS produces the same hash. If you hash the same file on Monday and Friday, you get the identical 64-character string.

Property 2 - Avalanche effect: Change even ONE bit of the input and the hash changes COMPLETELY. Not just one character - the entire 64-character string becomes unrecognizable. This makes tampering detectable.

How the Hash Chain Works

We do not just hash the source file once. We build a **hash chain**: at every step of the pipeline, we compute a new hash that combines the PREVIOUS hash with the current data. This is like blockchain technology:

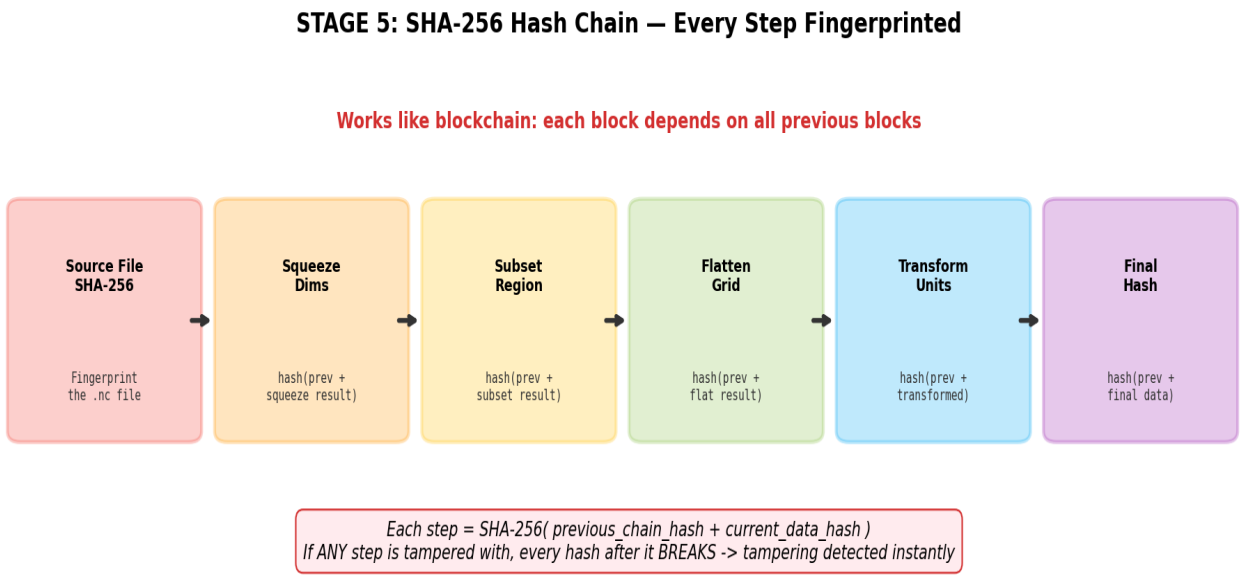


Figure 3: SHA-256 hash chain - each step depends on all previous steps

Step 0: hash = SHA-256(source_file). Step 1: hash = SHA-256(step0_hash + squeeze_result). Step 2: hash = SHA-256(step1_hash + subset_result). And so on. If someone tampers with the source file, EVERY hash in the chain breaks. If someone tampers with the transformation at step 3, every hash from step 3 onward breaks. You can verify the entire chain by replaying it.

In our pipeline, each dataset gets its own hash chain. DS01 has 6 steps (ingest + 5 transforms). DS02 has 9 steps (ingest + squeeze + subset + flatten + 4 transforms + 1 synthetic). DS03 has 6 steps (ingest + 5 transforms). All chains are saved to integrity_records.json.

Chapter 5: Stage 3 - Preprocessing

Making the NOAA Satellite Grid Manageable

Datasets 01 and 03 are already in tabular format (each row = one observation). They need no preprocessing. But Dataset 02 (NOAA) is a massive 4D grid that requires 5 preprocessing steps before it can be harmonized. The YAML config drives all of this:

```
preprocessing:
  squeeze_dims: ["altitude"]
  subset_region: {lat_min: 44.0, lat_max: 52.0, lon_min: -132.0, lon_max: -122.0}
  flatten_grid: true
  drop_nan_temperature: true
  drop_extreme_temperature: true
```

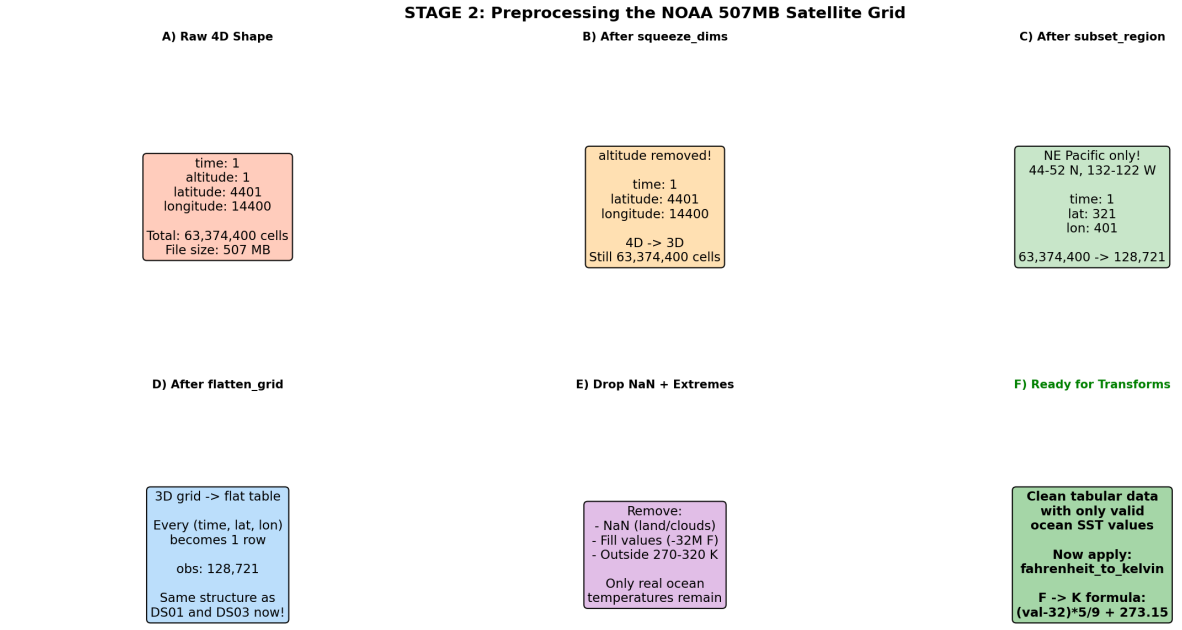


Figure 4: The 6-step preprocessing chain for the NOAA satellite grid

Step A - squeeze_dims: The NOAA file has 4 dimensions: (time=1, altitude=1, latitude=4401, longitude=14400). The altitude dimension has only 1 value (0.0 = sea surface). We "squeeze" it out, reducing from 4D to 3D. This is like removing a column from a spreadsheet that only has one value.

Step B - subset_region: After squeezing, we still have 4,401 x 14,400 = 63.4 million grid points covering the entire globe. We only care about the NE Pacific (44-52 N, 132-122 W). The subset reduces this to roughly 321 lat x 399 lon = 128,079 grid points per timestep. This is the step that prevents an out-of-memory crash.

Step C - flatten_grid: Now we have (time=1, lat=321, lon=399). We stack all 3 dimensions into a single "obs" dimension. Each combination of (time, lat, lon) becomes one row. Result: 128,079 flat rows, same structure as DS01 and DS03.

Step D - drop_nan_temperature: Many grid cells are NaN (land masses, cloud-covered areas). We remove all rows where temperature is NaN.

Step E - drop_extreme_temperature: The NOAA file uses fill values like -32,767 F for missing data. After converting to Kelvin, anything outside 270-320 K (which is -3 C to 47 C) gets dropped. This removes physically impossible values.

Chapter 6: Stage 4 - Unit Transformations

Converting Everything to Kelvin

Now all 3 datasets are in tabular form. But they still speak different "languages" for temperature. The transform module applies the correct conversion based on what the YAML config says:

Dataset	Source Variable	Source Unit	Transform	Formula	Target
DS01 ONC	Temperature	Kelvin	identity	no change	sea_water_temperature (K)
DS02 NOAA	sst	Fahrenheit	fahrenheit_to_kelvin	(val-32)*5/9+273.15	sea_water_temperature (K)
DS03 OOI	sea_water_temp_01	Celsius	celsius_to_kelvin	val + 273.15	sea_water_temperature (K)
DS03 OOI	z (altitude)	meters +up	altitude_to_depth	-val	depth (meters +down)

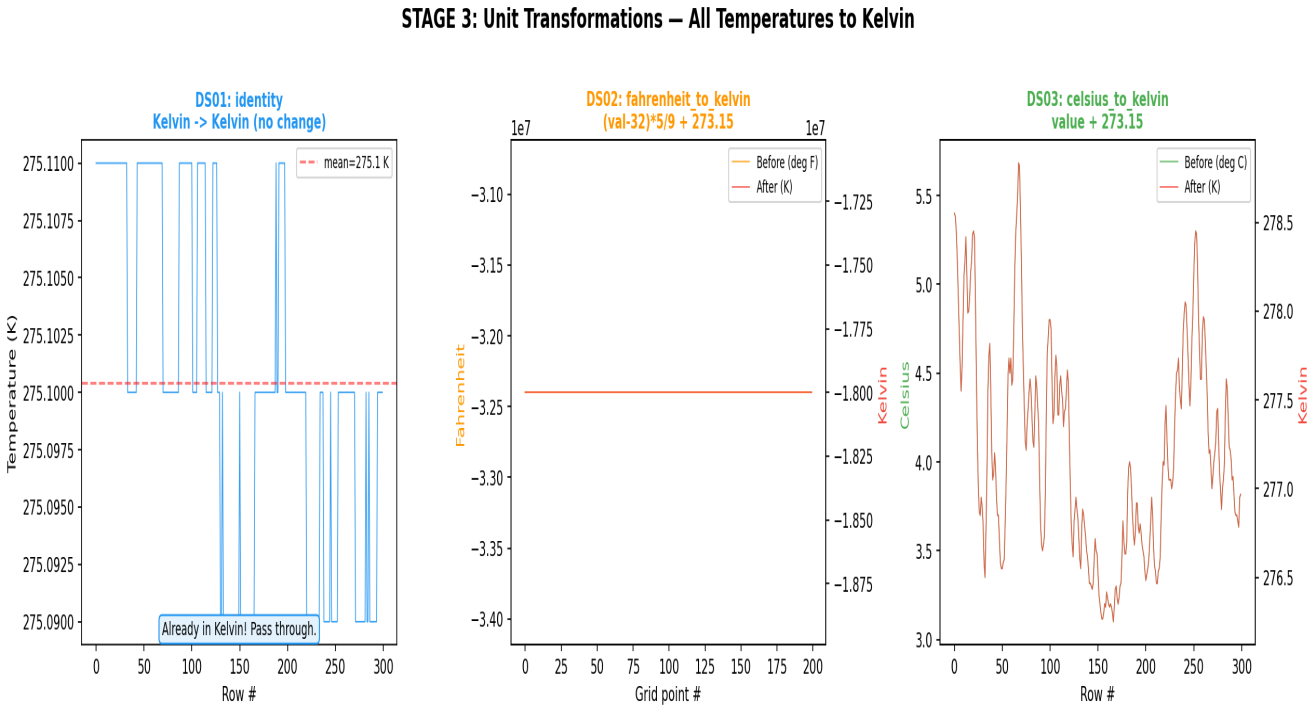


Figure 5: Before/after visualization of each unit transformation

Key insight: The pipeline code never says "if dataset is ONC, do identity." Instead, it reads the YAML config, looks up the transform name, finds the function in the Transform Registry, and applies it. This is what "config-driven" means. The transforms.py file contains 5 registered functions: identity, celsius_to_kelvin, fahrenheit_to_kelvin, altitude_to_depth, and ensure_utc.

Every transform also records: (1) what the input hash was, (2) what transform was applied, (3) what the output hash is, and (4) adds this as a new link in the SHA-256 hash chain. Plus, the W3C PROV provenance system logs it as an Activity with inputs and outputs.

Chapter 7: Stage 5 - Harmonization

One Schema to Rule Them All

After transformations, all temperatures are in Kelvin, all depths are in meters (positive down), all coordinates are WGS84, and all times are UTC. Now we map them to a **canonical schema**: 5 standard columns that every dataset must have, with standard names and units.

STAGE 4: After Harmonization — Canonical Schema (All Datasets Now Identical)

Column	Canonical Name	Units	Type	Example (DS01)	Example (DS02)	Example (DS03)
Temperature	sea_water_temperature	kelvin	float64	275.11	-17999692.85	278.55
Latitude	latitude	degrees_north	float64	47.93	48.00	45.93
Longitude	longitude	degrees_east	float64	-129.11	-128.50	-129.98
Depth	depth	meters (+down)	float64	1966.0	0.0 (synth)	0.0 (-z)
Time	time	UTC	datetime64	2024-06-03	2026-04-05	2026-04-06

5 columns, same names, same units, same types — ready for ML concat

Figure 6: The canonical schema - what every dataset looks like after harmonization

For DS02 (satellite data), there is no "depth" column because satellites measure the sea surface. The config specifies a **synthetic variable**: depth = 0.0 with the rationale "Satellite SST measures sea surface." This is recorded in provenance as a synthetic derivation (source = none).

After building the harmonized xarray Dataset, the validator checks every variable against the canonical schema's valid ranges. For sea_water_temperature, valid range is 270-320 K. For latitude, -90 to 90. For longitude, -180 to 180. For depth, 0 to 12,000 meters. Any violations are logged as warnings.

At this point, each dataset is a clean xarray Dataset with the exact same 5 columns. They are ready to be concatenated.

Chapter 8: Stage 6 - Writing to Zarr

ML-Optimized Compressed Storage

Now we combine all 3 harmonized datasets into one. The zarr_writer module does this:

Step 1 - Add tracking columns: Before combining, each dataset gets two extra columns:
- `source_dataset_idx`: integer ID (0=ONC, 1=NOAA, 2=OOI)
- `source_obs_index`: the original row number in that source dataset

These are the traceability columns. Given any row in the final Zarr, you can look up which source it came from and which exact row in the original file.

Step 2 - Concatenate: `xr.concat([ds01, ds02, ds03], dim='obs')` stacks all observations vertically. DS01's 1,020 rows, then DS02's ~71,770 rows, then DS03's 4,321 rows.

Step 3 - Write Zarr: The combined xarray Dataset is saved as a Zarr store. Zarr is a chunked, compressed array format. It uses zstd compression and stores data in chunks of ~38,556 observations. Unlike a single file, a Zarr store is a directory of small binary chunks. This means ML frameworks can read just the chunk they need without loading the whole dataset.

What is Zarr vs NetCDF? NetCDF stores data in a single monolithic file. Zarr stores it as a directory of small compressed chunks. For ML workloads that do random access, slicing, or parallel reading, Zarr is much faster. It also works natively with cloud storage (S3, GCS).

Step 4 - Save metadata: The Zarr store includes lineage_report.json (full provenance), integrity_records.json (all hash chains), and rich attributes on every variable.

Chapter 9: Stage 7 - Verification

Proving Nothing Was Tampered

After writing the Zarr store, the pipeline runs two critical verification steps:

Verification 1 - Source Integrity: Re-compute the SHA-256 hash of every source .nc file and compare it to the hash recorded during ingestion. If they match, the source files are INTACT. If any file was modified (even 1 byte), the hash would be completely different and it would report MODIFIED or MISSING.

Verification 2 - Hash Chain Validation: For each dataset's hash chain, replay every step: take step 0's hash, combine with step 1's data hash, verify it matches step 1's recorded chain hash. Then combine step 1's chain hash with step 2's data hash, verify step 2. And so on. If ALL steps match, the chain is VALID. If any step was altered, the chain breaks.

```
In our pipeline run:
- dataset_01_onc_endeavour.nc: INTACT, chain VALID (6 steps)
- dataset_02_noaa_sst_gridded.nc: INTACT, chain VALID (9 steps)
- dataset_03_ooi_vent.nc: INTACT, chain VALID (6 steps)
```

All verification results, the full YAML config used, and the pipeline run timestamp are saved to pipeline_run_log.json. This means months from now, anyone can open this log and know exactly what config produced what output, when, and verify it is untampered.

Chapter 10: The Complete Pipeline Flow

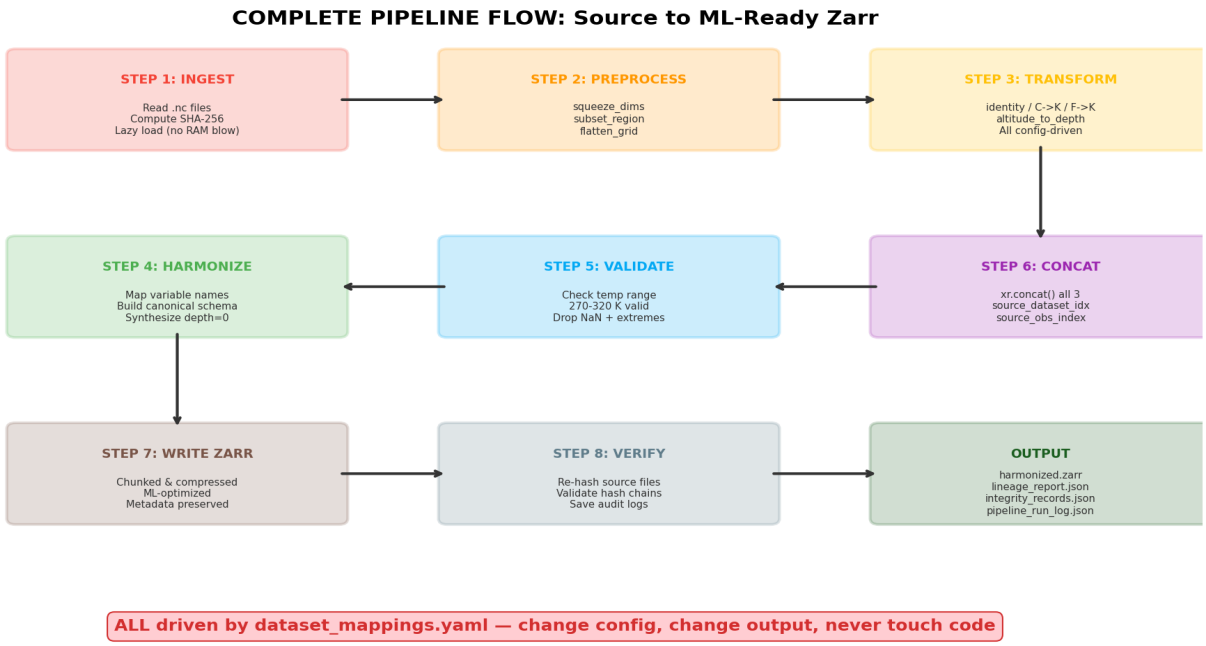


Figure 7: Complete 8-step pipeline from source ingestion to ML-ready output

The pipeline orchestrator (run_pipeline.py) executes all 8 steps in order. Each step prints detailed progress to the console. The entire flow is: Load Config -> Ingest (SHA-256 + lazy load) -> Harmonize (preprocess + transform + validate) -> Write Zarr -> Verify Sources -> Verify Hash Chains -> Save Audit Logs -> Demo Trace. A single command runs everything: `python3 run_pipeline.py`

Chapter 11: Re-Derivability

Change Config, Not Code

Re-derivability means: given the same source files, you can produce a different output by changing only the YAML config file. No Python code needs to change. This is a core design constraint.

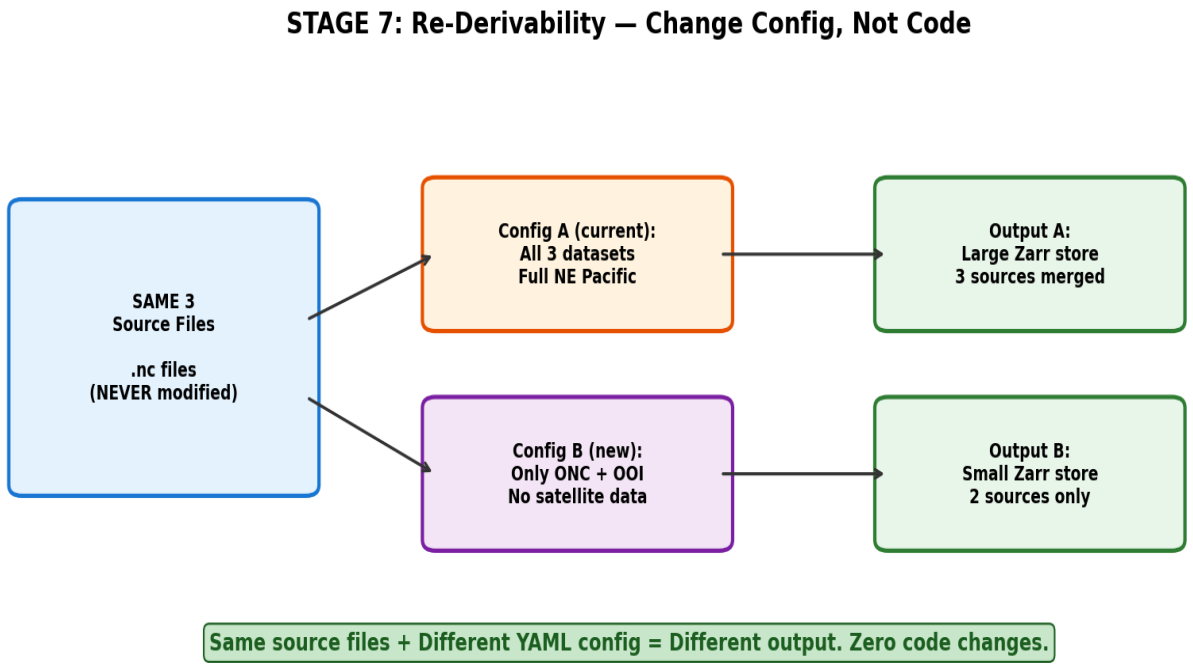


Figure 8: Same sources, different config = different output

Real Scenario 1: A researcher says "I only want the CTD sensors, not satellite data." You open dataset_mappings.yaml, delete the dataset_02 block, and re-run. The pipeline now produces a Zarr store with only 5,341 observations from DS01 + DS03. No code touched.

Real Scenario 2: A colleague says "I need the temperature in Celsius, not Kelvin." You edit the transform for each dataset from 'celsius_to_kelvin' to 'identity' and change target_units to celsius. Re-run. Done. The hash chains will be different (proving a different config was used), but the provenance log records exactly what changed.

Real Scenario 3: You want a larger spatial region (30-60 N instead of 44-52 N). Edit subset_region in the YAML. The pipeline will subset differently, producing more rows. Same code, same source files, different config, different result.

Chapter 12: What the ML Engineer Gets

The Final Zarr Store

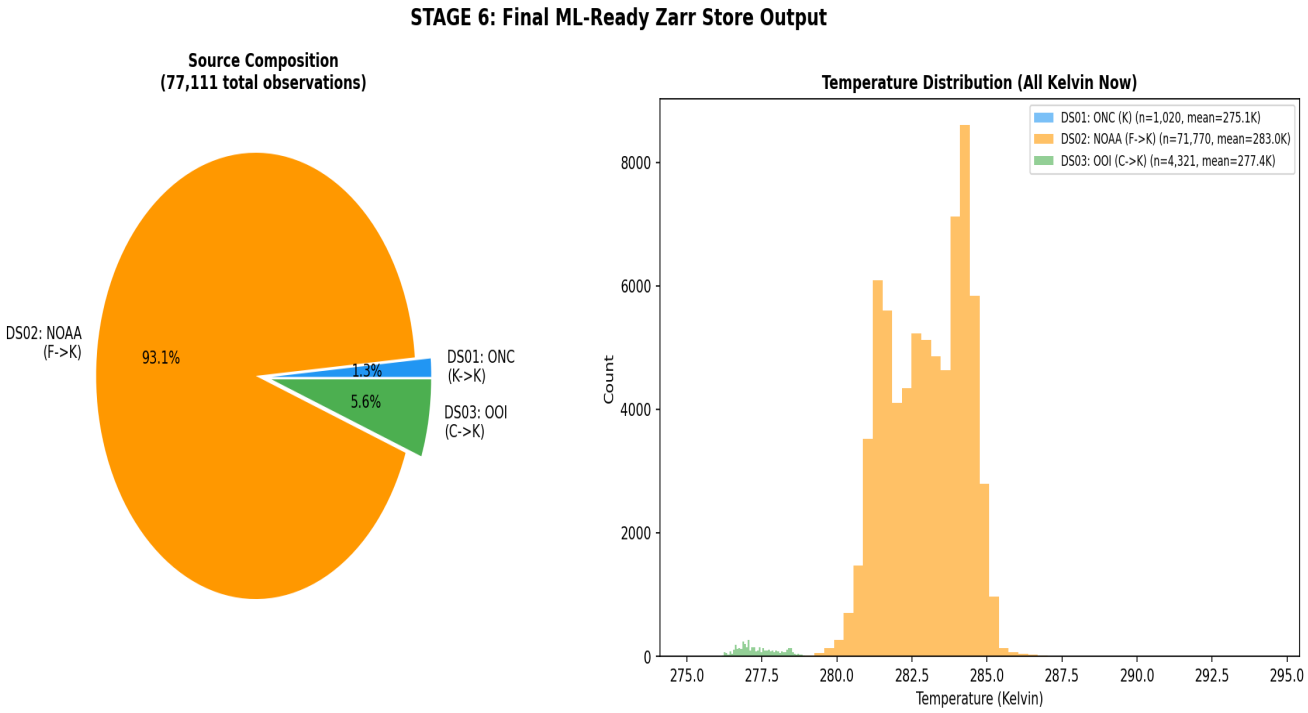


Figure 9: Source composition and temperature distribution in the final Zarr store

The final Zarr store contains **77,111 observations** from 3 sources: 1,020 from ONC (1.3%), 71,770 from NOAA (93.1%), and 4,321 from OOI (5.6%). All temperatures are in Kelvin, ranging from 275.07 K to 294.46 K. NOAA dominates because satellite data covers a grid of thousands of spatial points, while the CTD sensors are single-point measurements.

What the ML Engineer Sees: The Final Zarr Store

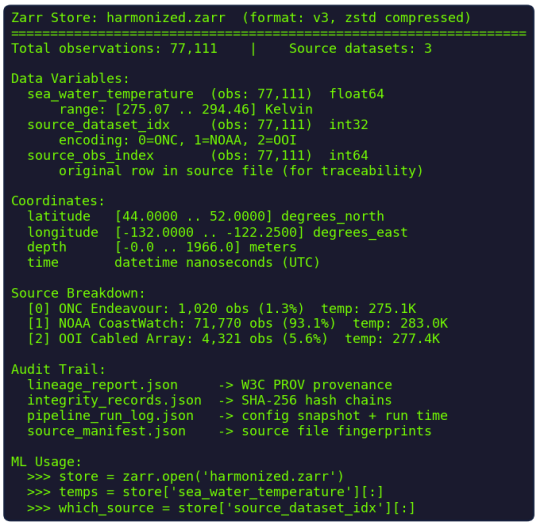


Figure 10: The Zarr store contents as an ML engineer would see them

Traceability in action: Take row #5000 in the Zarr store. Its source_dataset_idx is 1 (NOAA) and source_obs_index is 3980. That means it came from the 3,980th observation after flattening and cleaning the NOAA grid. The lineage_report.json tells you the original variable was 'sst', the unit was Fahrenheit, and the transform 'fahrenheit_to_kelvin' was applied. The integrity_records.json proves the hash chain is unbroken.

Chapter 13: The Guiding Question Answer

"In a pipeline that ingests immutable source data and produces harmonized, ML-ready outputs, how should the system separate the act of ingestion (fixed, append-only) from the act of interpretation (configurable, re-derivable)?"

The Answer:

Ingestion is separated from interpretation through a **YAML configuration layer**. The ingestor reads source files exactly as they are and records their SHA-256 fingerprints. It knows nothing about variable names, units, or transformations. The harmonizer then reads the YAML config to learn what each variable means and how to convert it. This separation means:

1. Ingestion is fixed: The same .nc file always produces the same hash, the same raw xarray Dataset. It is append-only: new files can be added but existing ones never change.

2. Interpretation is configurable: What "Temperature" means (which column, which units, which transform) is defined entirely in the YAML config. Change the config and the interpretation changes.

3. Re-derivability is guaranteed: Because the config is saved in the pipeline run log, you can always reproduce any past output by using the same config with the same source files.

4. Traceability is built-in: The SHA-256 hash chain proves data integrity. The W3C PROV lineage proves exactly which variable, from which file, via which transform, became each output column.

PROJECT FILE STRUCTURE

```
Capstone_Project/
  run_pipeline.py           # Main orchestrator (8 steps)
  pipeline/configs/
    dataset_mappings.yaml    # THE config file (drives everything)
  pipeline/modules/
    ingestor.py             # Read + SHA-256 (immutable)
    harmonizer.py           # Preprocess + transform (config-driven)
    transforms.py           # 5 registered transform functions
    integrity.py            # SHA-256 hash chains
    provenance.py           # W3C PROV lineage tracking
    zarr_writer.py          # Concat + write Zarr + metadata
  source_data/               # 3 immutable .nc files
  output/zarr_store/         # ML-ready Zarr + audit files
```

End of Document